



Обучение с подкреплением

Страшная математика + нестрашная физика жидкостей

Леонтьев Михаил

ПАДИИ, 2 курс

НИУ ВШЭ — Санкт-Петербург

Содержание

- 1 Введение в RL
- 2 Градиентный спуск
- 3 REINFORCE
- 4 Actor-Critic
- 5 Proximal Policy Optimization
- 6 Пример



Введение в RL

Обучение с подкреплением

Представьте, что вам нужно научиться
(=написать алгоритм) для игры в Марио.
У вас есть: приставка с этой игрой и — В
С Ё.
Как быть?

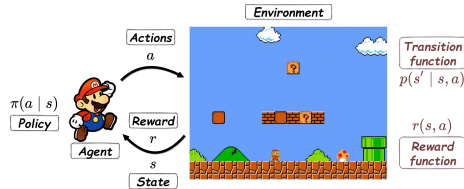


Цикл взаимодействия

В каждый момент времени t :

- Наша среда находится в состоянии (state) s_t
- Мы можем сделать действие (action) a_t
- За это действие мы получаем награду (reward) r_t (в зависимости от s_t, a_t)
- После этого игра переходит в состояние s_{t+1} (в зависимости от s_t, a_t)

Есть ли в этой схеме какие-то допущения?

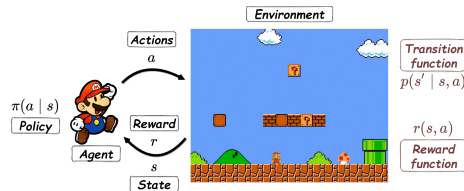


Цикл взаимодействия

В каждый момент времени t :

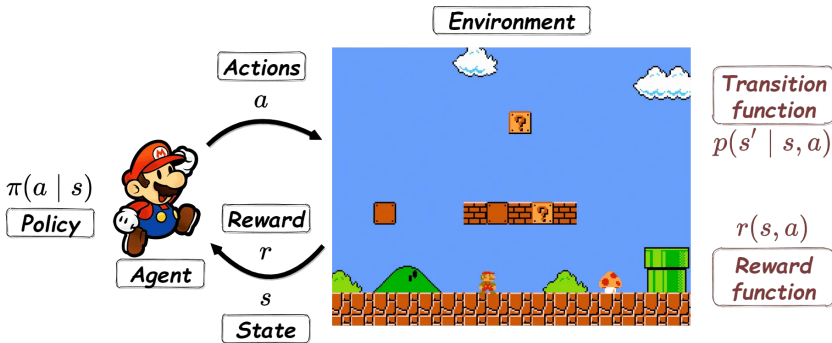
- Наша среда находится в состоянии (state) s_t
- Мы можем сделать действие (action) a_t
- За это действие мы получаем награду (reward) r_t (в зависимости от s_t, a_t)
- После этого игра переходит в состояние s_{t+1} (в зависимости от s_t, a_t)

Есть ли в этой схеме какие-то допущения?
Марковское свойство.



Policy

«Игроком» в данном случае является $\pi(a_t | s_t)$ (**policy**) — насколько хорошо в данном состоянии s_t выполнить действие a_t (число $[0, 1]$ — вероятность).



Примеры

Задача	Action Space	State Space	Reward
Марио			
Беспилотная машина			
Шахматы			

Примеры

Задача	Action Space	State Space	Reward
Марио	Дискретные: Кнопки джой-стика	Кадр экрана, позиции врагов и объектов, скорость персонажа	+ за прогресс и очки, - за смерть/урон
Беспилотная машина			
Шахматы			

Примеры

Задача	Action Space	State Space	Reward
Марио	Дискретные: Кнопки джой-стика	Кадр экрана, позиции врагов и объектов, скорость персонажа	+ за прогресс и очки, - за смерть/урон
Беспилотная машина	Непрерывные: руль, газ, тормоз	Камеры, LiDAR, скорость, разметка, положение на карте	+ за безопасный километраж, - за столкновения и нарушения
Шахматы			

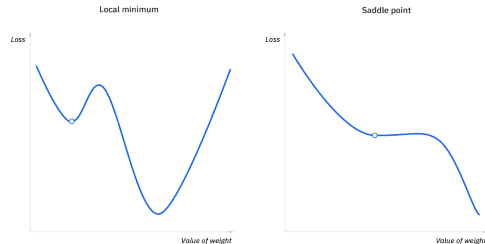
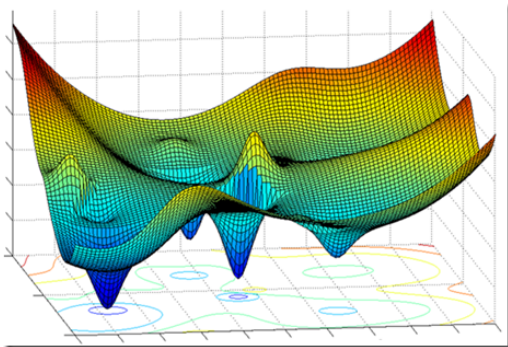
Примеры

Задача	Action Space	State Space	Reward
Марио	Дискретные: Кнопки джой-стика	Кадр экрана, позиции врагов и объектов, скорость персонажа	+ за прогресс и очки, - за смерть/урон
Беспилотная машина	Непрерывные: руль, газ, тормоз	Камеры, LiDAR, скорость, разметка, положение на карте	+ за безопасный километраж, - за столкновения и нарушения
Шахматы	Дискретные: любой легальный ход	Позиция фигур на доске, очередь хода, спецправа (рокировка, взятие на проходе)	+1 победа, 0 ничья, -1 поражение (бонус: за количество ходов)



Градиентный спуск

Постановка задачи



Проблема: как найти минимум функции $\mathcal{J}(\theta)$, если мы умеем только считать её в одной точке (особенно, если считать её долго)?

Вопрос: что показывает производная в точке?

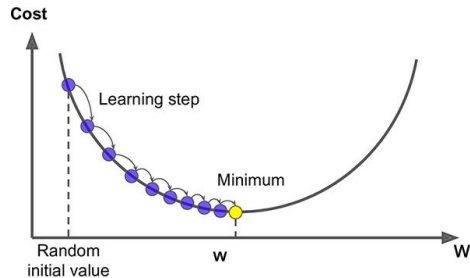
Градиентный спуск

Заметим, что направление, в котором функция убывает быстрее всего — это $-\mathcal{J}'$ (далее будем писать $-\nabla_{\theta}\mathcal{J}$).

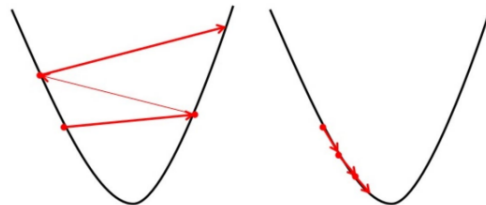
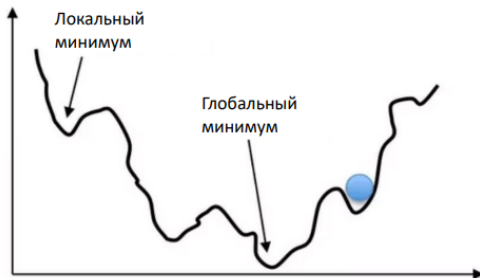
Тогда выберем случайную точку $\theta^{(0)}$ и будем обновлять её

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla_{\theta} \mathcal{J}(\theta^{(k)})$$

, где $\alpha > 0$ — скорость спуска (**learning rate**).



Проблемы



Расходимость или медленная
сходимость



REINFORCE

Policy-based

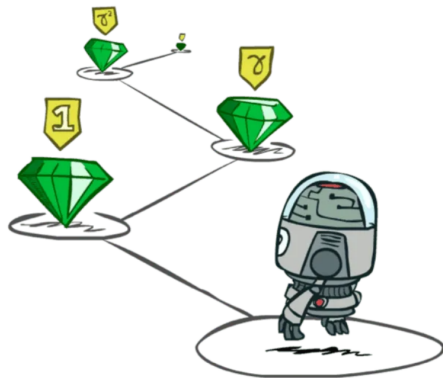
Пусть π_θ — какая-то *модель машинного обучения* с параметрами θ , которая по состоянию s_t выдаёт вероятности действия $\pi_\theta(a_t|s_t)$.

Что оцениваем

Пусть мы сыграли один раз, получив траекторию $\tau = (s_t, r_t, a_t)_{t \geq 0}$. Оценим её так:

$$R(\tau) = \sum_{t=0}^{+\infty} \gamma^t r_t$$

, где $\gamma \in [0, 1]$ — коэффициент дисконтирования (**discounting**).



Exploration vs. Exploitation



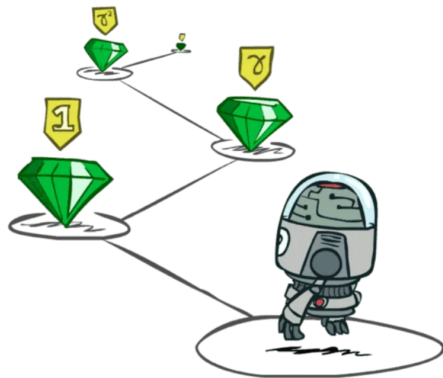
Какая стратегия здесь оптимальна?

Что оцениваем

Пусть мы сыграли один раз, получив траекторию $\tau = (s_t, r_t, a_t)_{t \geq 0}$. Оценим её так:

$$R(\tau) = \sum_{t=0}^{+\infty} \gamma^t r_t$$

, где $\gamma \in [0, 1]$ — коэффициент дисконтирования (**discounting**).



Тогда как оценить «успешность» нашей π_θ ?

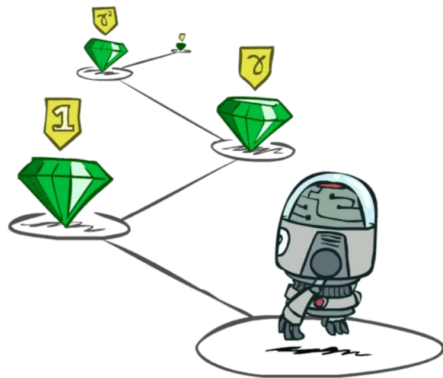
Что оцениваем

Пусть мы сыграли один раз, получив траекторию $\tau = (s_t, r_t, a_t)_{t \geq 0}$. Оценим её так:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \sum_{t=0}^{+\infty} \gamma^t r_t$$

, где $\gamma \in [0, 1]$ — коэффициент дисконтирования (**discounting**).

Тогда как оценить «успешность» нашей π_θ ?



Градиентный спуск

Мы хотим максимизировать $\mathcal{J}(\theta)$ по всем θ . Можем использовать *град. подъем* для этого. Но тогда нужно уметь считать $\nabla_{\theta}\mathcal{J}$.

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)] = \sum_{\tau} \mathbb{P}(\tau) \cdot R(\tau)$$

Однако считать $\mathbb{P}(\tau)$ может быть сложно считать. *Что делать?*



Градиентный спуск

Мы хотим максимизировать $\mathcal{J}(\theta)$ по всем θ . Можем использовать *град. подъем* для этого. Но тогда нужно уметь считать $\nabla_{\theta} \mathcal{J}$.

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)] = \sum_{\tau} \mathbb{P}(\tau) \cdot R(\tau)$$

Однако считать $\mathbb{P}(\tau)$ может быть сложно считать. *Что делать?*

Policy Gradient Theorem

$$\nabla_{\theta} \mathcal{J} = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot R(\tau)]$$



REINFORCE

Алгоритм **REINFORCE** (или Monte-Carlo Reinforce):

- 1 Запустить N проходов для нашей $\pi_{\theta^{(k)}}$
- 2 Для каждой из получившихся траекторий τ_j посчитать
$$\hat{g}_j = \sum_t \nabla_{\theta} \log \pi_{\theta^{(k)}}(a_t | s_t) \cdot R(\tau)$$
- 3 Тогда посчитаем $\nabla_{\theta} J(\theta^{(k)}) = \frac{1}{N} \sum_{j=1}^N \hat{g}_j$
- 4 Обновляем полиси $\theta^{(k+1)} = \theta^{(k)} + \alpha \nabla_{\theta} J(\theta^{(k)})$

Минусов нет. *Кроме...*

Недостатки REINFORCE

- 1 Нужно делать полный проход много раз
- 2 Мы полностью теряем информацию между $\theta^{(k)} \rightarrow \theta^{(k+1)}$
- 3 Если мы случайно пойдём не в том направлении, то мы испортим себе входные данные.



Actor-Critic

Value function

Проблема: $R(\tau)$ оценивает не только успешность нашей полиси, но и насколько удачны состояния, которые он посещает.

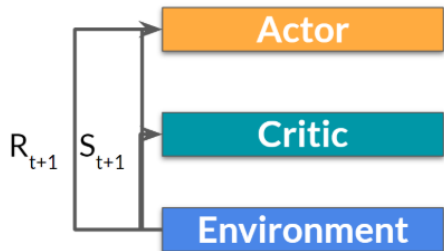
$$V^\pi(s) = \mathbb{E}_\pi[r_1 + \gamma r_2 + \gamma^2 r_3 + \dots | s_0 = s]$$

, то есть эта функция показывает, насколько удачно для нашей полиси π состояние s .

Critic model

Давайте вместе со стратегией π_θ обучать ещё и V_φ (то есть научимся понимать, насколько хороший reward мы умеем получать из такого состояния).

Как теперь хочется изменить функцию $R(\tau)$?



Critic model

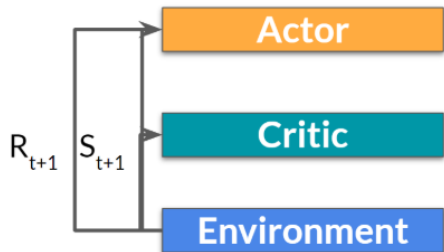
Давайте вместе со стратегией π_θ обучать ещё и V_φ (то есть научимся понимать, насколько хороший reward мы умеем получать из такого состояния).

Как теперь хочется изменить функцию $R(\tau)$?

Введем функцию **advantage** (преимущества):

$$A_t(\tau) = \boxed{R(\tau)} - V_\varphi(s_t)$$

$$A_t(\tau) = \boxed{(r_t + \gamma V_\varphi(s_{t+1}))} - V_\varphi(s_t)$$



Advantage Actor Critic

Плюсы

- 1 Более точно измеряем качество полиси
- 2 Сохраняем информацию для V_ϕ
- 3 Можно не прогонять модель до конца

Минусы

- 1 Высокая чувствительность к *learning rate*
- 2 Теряем информацию для π_θ

если есть ко мне претензии
подойдите к ближайшему
дереву и выскажите
ему всё

мне всё передадут



Proximal Policy Optimization

Проблематика

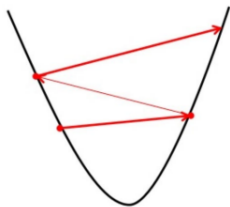
$$\nabla_{\theta} \mathcal{J} = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A_t]$$

Что происходит, если мы делаем большой шаг? А если маленький?

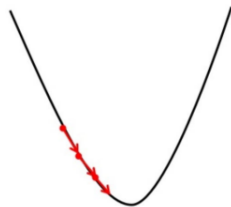
Проблематика

$$\nabla_{\theta} \mathcal{J} = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A_t]$$

Что происходит, если мы делаем большой шаг? А если маленький?



Большая скорость обучения:
метод расходится
(ошибка увеличивается)



Малая скорость обучения:
метод медленно сходится



Importance Sampling

А что если использовать данные от **старой** политики π_{θ_k} , чтобы оценить новую π_{θ} ?

Каждое действие нужно взвесить: насколько новая политика отличается от старой в этой точке:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$$

- $r_t > 1$ — новая политика делает это действие *чаще*
- $r_t < 1$ — новая политика делает это действие *реже*
- $r_t = 1$ — политики совпадают в этой точке

Тогда будем оптимизировать:

$$\mathcal{L}(\theta) = \mathbb{E}[r_t(\theta) \cdot A_t]$$

Importance Sampling

Проблема: без ограничений $r_t(\theta)$ может стать очень большим.

Если действие оказалось удачным ($A_t > 0$) — алгоритм захочет резко увеличить его вероятность, $r_t \gg 1$.

Снова получаем большой шаг. *Что делать?*



PPO-Clip

Идея PPO: просто **обрежем** $r_t(\theta)$ — не дадим ему уходить далеко от 1:

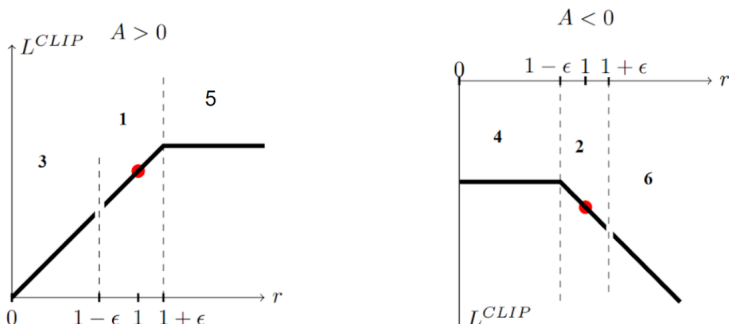
$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E} [\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot A_t)]$$

где ε — гиперпараметр (обычно 0.1–0.2).

Разберём два случая:

- $A_t > 0$: действие хорошее, хотим увеличить r_t . Если $r_t > 1 + \varepsilon$ — $\min$ выбирает clip , градиент **обнуляется**.
- $A_t < 0$: действие плохое, хотим уменьшить r_t . Если $r_t < 1 - \varepsilon$ — снова clip , обновления **нет**.

PPO-Clip: график



Вывод: PPO никогда не вознаграждает политику за то, что она слишком сильно отклонилась от предыдущей — ни в хорошую, ни в плохую сторону.

Алгоритм PPO

- 1 Собрать N траекторий с текущей политикой $\pi_{\theta(k)}$
- 2 Посчитать advantage A_t для каждого шага
- 3 Несколько раз обновить θ, φ по одним и тем же данным:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}^{\text{CLIP}}(\theta)$$

- 4 Повторить

Ключевое отличие от REINFORCE/A2C: шаг 3 — мы используем одни данные **несколько раз**, пока clip не начинает срабатывать слишком часто.



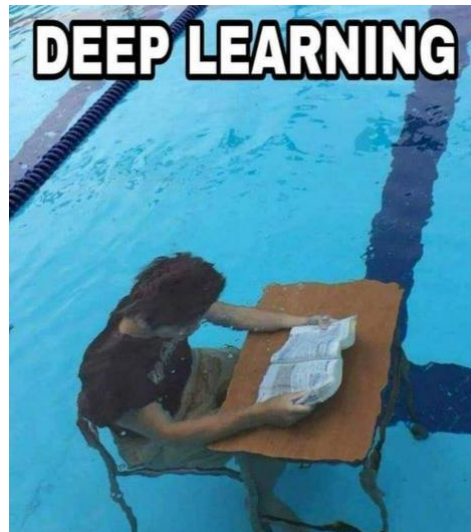
Пример

Постановка задачи

Дано: подземный нефтяной пласт. Нужно решить:

- Сколько скважин бурить?
- Какого типа — добывающие или нагнетательные?
- Где бурить?
- В каком *порядке*?

Цель: максимизировать суммарную прибыль с учётом затрат и дисконтирования.



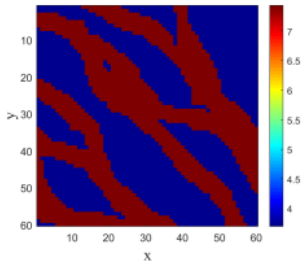
Почему это сложно

- Решения **взаимозависимы**: скважина здесь меняет давление там
- Решения **последовательны**: каждый этап бурения зависит от предыдущих
- Среда **дорогая**: одна симуляция пласта — часы вычислений
- Пространство решений **огромное**: 60×60 сетка, 5 этапов бурения

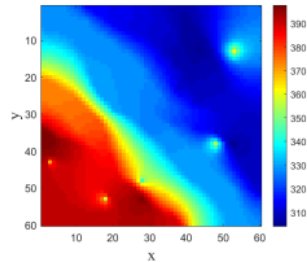
Формализация как MDP

Компонент	В задаче
State s_t	4 карты 60×60 (проницаемость, давление, насыщенность, скважины) + номер этапа + счётчики скважин
Action a_t	(1) бурить/нет и тип скважины; (2) если бурить — клетка на сетке
Reward r_t	прирост NPV на данном этапе бурения
Episode	5 этапов бурения → терминальное состояние
Policy π	нейросеть: карты → решение о бурении

Состояние: 4 карты пласта



(a) Permeability map (in log scale)



(b) Pressure map

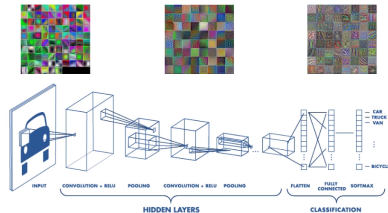
Почему CNN?

Состояние агента — это **4 карты**, то есть пространственные данные.

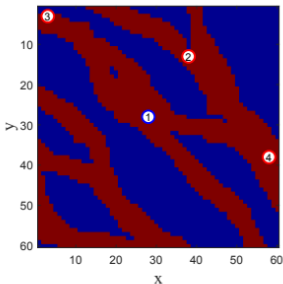
Сверточная нейросеть (CNN) умеет находить *локальные паттерны* на картах: где высокая проницаемость рядом с уже пробуренной скважиной?

Обычная нейросеть «не знает», что соседние клетки связаны. CNN — знает.

Convolutional Neural Network

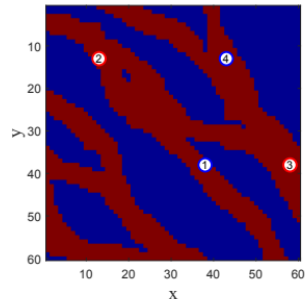


Результаты: решения агента



(a) Small network (\$249.80 million)

Малая сеть: \$249.8 млн



(b) Large network (\$271.86 million)

Большая сеть: \$271.9 млн

Сравнение с классическим методом

Метод	NPV	Скважин	Результат
PSO-MADS	\$272.1 млн	5	одно решение
PPO (малая)	\$249.8 млн	4	политика
PPO (большая)	\$271.9 млн	4	политика

PSO-MADS тратит 10 000 симуляций на **одно** решение для **одного** пласта.

PPO получает **политику** — функцию, которая работает при любом состоянии.

Спасибо за внимание!

Леонтьев Михаил • НИУ ВШЭ — Санкт-Петербург

Telegram



https://t.me/michlea_tg

Фидбек+материалы



<https://forms.gle/uQ1pAMNS5AStLQVEA>